

ui.log 全面详解

`ui.log` 是 NiceGUI 提供的轻量级日志展示组件，专为在 Web 应用中实时呈现日志信息设计，支持日志分级、过滤、清空、自动滚动等核心功能，适用于调试输出、操作记录、系统状态监控等场景。以下从核心特性、配置参数、使用方法、进阶功能等维度展开全面解析。

一、核心定位与基础特性

1. 核心功能

- 日志分级显示：支持 `DEBUG` / `INFO` / `WARNING` / `ERROR` / `CRITICAL` 五级日志，不同级别自动应用差异化样式（颜色、图标）。
- 实时日志追加：支持通过代码动态添加日志条目，自动滚动到底部（可配置关闭）。
- 日志过滤：内置级别过滤功能，可快速筛选指定级别的日志。
- 交互操作：支持清空日志、复制单条日志、复制全部日志。
- 样式定制：支持自定义日志条目样式、时间格式、是否显示图标等。
- 性能优化：支持日志容量限制，自动删除 oldest 日志，避免内存溢出。

2. 基础用法

通过 `ui.log()` 创建日志组件，直接调用 `log.debug()` / `log.info()` 等方法添加对应级别日志，示例代码结构如下：

```
from nicegui import ui

# 创建日志组件（默认显示所有级别，自动滚动）
log = ui.log()

# 添加不同级别的日志
log.debug('调试信息：初始化完成')
log.info('普通信息：用户登录成功')
log.warning('警告信息：磁盘空间不足')
log.error('错误信息：数据库连接失败')
log.critical('严重错误：系统即将关闭')

ui.run()
```

二、关键配置参数

`ui.log` 的初始化参数用于定义日志组件的显示规则、交互行为和容量限制，以下是完整参数说明：

参数名	类型	说明	默认值	版本特性
max_lines	整数	日志最大条目数，超过后自动删除最早的日志	1000	-
auto_scroll	布尔值	新增日志时是否自动滚动到底部	True	-

参数名	类型	说明	默认值	版本特性
level	字符串/int	默认显示的日志级别（过滤门槛）：- 字符串： <code>DEBUG / INFO / WARNING / ERROR / CRITICAL</code> ；- 整数：0=DEBUG、1=INFO、2=WARNING、3=ERROR、4=CRITICAL	<code>DEBUG</code> (0)	-
show_levels	布尔值	是否显示日志级别标签（如 [INFO]）	True	-
show_times	布尔值	是否显示日志时间戳	True	-
show_icons	布尔值	是否显示日志级别对应的图标	True	-
time_format	字符串	时间戳格式（遵循 Python <code>datetime.strftime</code> 语法）	"%Y-%m-%d %H:%M:%S"	-
expandable	布尔值	是否允许通过点击展开 / 折叠日志组件	False	2.0.0 版本新增
collapsed	布尔值	初始状态是否折叠（仅 <code>expandable=True</code> 时生效）	False	2.0.0 版本新增
tailwind	字符串	自定义 Tailwind CSS 类，用于修改组件整体样式	""	-
html_id	字符串	组件 DOM 元素的 ID，用于自定义 CSS/JS 操作	None	2.16.0 版本新增

日志级别对应关系

级别名称	整数标识	颜色样式	图标
DEBUG	0	灰色	
INFO	1	蓝色	
WARNING	2	黄色	
ERROR	3	红色	
CRITICAL	4	深红色	

三、核心功能用法

1. 日志级别控制

(1) 初始化时指定默认级别

仅显示指定级别及更高级别的日志（如默认显示 `WARNING` 及以上）：

```
# 仅显示 WARNING、ERROR、CRITICAL 级别日志
log = ui.log(level='WARNING')
log.debug('调试信息：不会显示')
log.warning('警告信息：会显示')
log.error('错误信息：会显示')
```

(2) 动态修改过滤级别

通过 `log.level` 属性或 `set_level()` 方法动态切换过滤级别：

```
log = ui.log()

# 按钮控制级别切换
with ui.row():
    ui.button('显示所有级别', on_click=lambda: log.set_level('DEBUG'))
    ui.button('仅显示错误及以上', on_click=lambda: log.set_level('ERROR'))
    ui.button('当前级别', on_click=lambda: ui.notify(f'当前级别: {log.level}'))
```

2. 日志追加与格式化

(1) 基础日志追加

除了分级方法（`debug()` / `info()` 等），还可通过 `push()` 方法直接添加日志，支持自定义级别：

```
log = ui.log()

# 直接添加 INFO 级日志（等价于 log.info()）
log.push('普通信息', level='INFO')
# 添加自定义级别（需指定整数标识，如 2=WARNING）
log.push('自定义警告', level=2)
```

(2) 格式化日志

支持使用 f-string、`format()` 等 Python 格式化语法，添加带变量的日志：

```
username = 'admin'
login_time = '2024-05-20 14:30:00'

log = ui.log()
log.info(f'用户 [{username}] 于 {login_time} 登录系统')
log.error(f'用户 [{username}] 尝试访问未授权资源')
```

3. 交互操作

(1) 清空日志

通过 `log.clear()` 方法清空所有日志条目：

```
log = ui.log()
log.info('测试日志 1')
log.info('测试日志 2')

ui.button('清空日志', on_click=log.clear)
```

(2) 复制日志

- 单条日志：点击日志条目右侧的「复制」图标，复制当前条目内容。
- 全部日志：通过 `log.copy_all()` 方法复制所有日志（可绑定到按钮）：

```
log = ui.log()
log.info('日志条目 1')
log.info('日志条目 2')

ui.button('复制所有日志', on_click=lambda: log.copy_all() and ui.notify('已复制所有日志'))
```

(3) 自动滚动控制

通过 `auto_scroll` 参数或 `set_auto_scroll()` 方法控制是否自动滚动：

```
log = ui.log(auto_scroll=True)

# 开关控制自动滚动
ui.switch('自动滚动', value=True).bind_value_to(log, 'auto_scroll')
```

4. 容量限制与日志清理

通过 `max_lines` 参数限制日志最大条目数，超过后自动删除最早的日志，避免组件体积过大：

```
# 最多保留 100 条日志，超过自动清理 oldest
log = ui.log(max_lines=100)

# 批量添加 200 条日志，最终仅保留最后 100 条
for i in range(200):
    log.info(f'日志条目 {i+1}')
```

5. 样式定制

(1) 自定义时间格式

通过 `time_format` 参数修改日志时间戳的显示格式（遵循 Python `datetime.strftime` 语法）：

```
# 显示到毫秒，格式：2024-05-20 14:30:00.123
log = ui.log(time_format="%Y-%m-%d %H:%M:%S.%f"[:-3])
log.info('带毫秒的日志')
```

(2) 隐藏级别 / 时间 / 图标

通过 `show_levels` / `show_times` / `show_icons` 参数控制日志元素的显示：

```
# 隐藏图标和级别，仅显示时间和日志内容
log = ui.log(show_icons=False, show_levels=False)
log.info('仅显示时间和内容的日志')
```

(3) 自定义 Tailwind 样式

通过 `tailwind` 参数为组件添加自定义样式（如修改背景色、字体大小）：

```
# 深色背景、白色文字、圆角边框
log = ui.log(tailwind='bg-gray-800 text-white rounded-lg p-4')
log.info('深色模式日志')
log.error('错误信息在深色模式下显示')
```

6. 折叠功能 (2.0.0+)

通过 `expandable=True` 启用组件折叠功能，初始状态可通过 `collapsed` 参数控制：

```
# 启用折叠，初始状态为折叠
log = ui.log(expandable=True, collapsed=True)
log.info('折叠状态下需点击展开才能看到日志')

# 按钮控制折叠/展开
ui.button('切换折叠状态', on_click=log.toggle)
```

四、进阶用法

1. 绑定到 Python 标准日志模块

将 `ui.log` 作为 Python 标准 `logging` 模块的处理器，实现日志的统一管理（同时输出到控制台和 Web 界面）：

```
import logging
from nicegui import ui

# 创建 ui.log 组件
log = ui.log()
```

```

# 定义自定义日志处理器
class UILogHandler(logging.Handler):
    def emit(self, record):
        # 根据日志级别调用对应的 ui.log 方法
        level = record.levelno
        message = self.format(record)
        if level == logging.DEBUG:
            log.debug(message)
        elif level == logging.INFO:
            log.info(message)
        elif level == logging.WARNING:
            log.warning(message)
        elif level == logging.ERROR:
            log.error(message)
        elif level == logging.CRITICAL:
            log.critical(message)

# 配置标准日志模块
logging.basicConfig(level=logging.DEBUG)
logger = logging.getLogger(__name__)
# 添加 UI 处理器
logger.addHandler(UILogHandler())

# 测试标准日志输出（会同时显示在控制台和 web 日志组件中）
logger.debug('标准日志 - 调试信息')
logger.info('标准日志 - 普通信息')
logger.warning('标准日志 - 警告信息')

ui.run()

```

2. 实时日志更新（结合异步任务）

在异步任务中动态添加日志，实现系统状态的实时监控（如文件上传进度、任务执行状态）：

```

import asyncio
from nicegui import ui

log = ui.log(auto_scroll=True)

# 异步任务：模拟实时输出日志
async def async_task():
    for i in range(10):
        await asyncio.sleep(1)
        log.info(f'异步任务执行中：进度 {i*10}%')
    log.success('异步任务执行完成！') # success 等价于 info，样式一致

# 启动异步任务
ui.button('启动异步任务', on_click=lambda: ui.run_javascript('window.scrollTo(0, document.body.scrollHeight)') or asyncio.create_task(async_task()))

ui.run()

```

3. 自定义日志条目样式

通过 `log.push()` 方法的 `classes` 参数，为单条日志添加自定义 Tailwind 类，实现差异化样式：

```
log = ui.log()

# 普通日志
log.info('普通信息日志')
# 自定义样式：绿色文字、加粗
log.push('成功信息（自定义样式）', level='INFO', classes='text-green-600 font-bold')
# 自定义样式：黄色背景、黑色文字
log.push('警告信息（自定义背景）', level='WARNING', classes='bg-yellow-200 text-black')
```

4. 日志导出

结合 `ui.download` 组件，实现日志的导出功能（导出为 TXT 文件）：

```
from nicegui import ui

log = ui.log()
log.info('日志条目 1')
log.warning('日志条目 2')
log.error('日志条目 3')

# 导出日志为 TXT 文件
def export_logs():
    # 收集所有日志条目（格式：时间 [级别] 内容）
    log_text = ''
    for entry in log.entries:
        time_str = entry['time'].strftime(log.time_format)
        level_str = entry['level'].upper()
        message = entry['message']
        log_text += f'{time_str} [{level_str}] {message}\n'
    # 创建下载链接
    ui.download(log_text, 'logs.txt', 'text/plain')

ui.button('导出日志', on_click=export_logs)

ui.run()
```

五、核心方法速查表

方法	说明	参数	示例
<code>debug(message)</code>	添加 DEBUG 级日志	<code>message</code> ：日志内容	<code>log.debug('调试信息')</code>
<code>info(message)</code>	添加 INFO 级日志	<code>message</code> ：日志内容	<code>log.info('普通信息')</code>
<code>warning(message)</code>	添加 WARNING 级日志	<code>message</code> ：日志内容	<code>log.warning('警告信息')</code>
<code>error(message)</code>	添加 ERROR 级日志	<code>message</code> ：日志内容	<code>log.error('错误信息')</code>

方法	说明	参数	示例
critical(message)	添加 CRITICAL 级日志	message: 日志内容	log.critical('严重错误')
success(message)	等价于 info, 样式一致	message: 日志内容	log.success('操作成功')
push(message, level=1, classes="")	通用日志添加方法	message: 内容; level: 级别 (0-4); classes: 自定义样式	log.push('自定义日志', level=2)
clear()	清空所有日志	-	log.clear()
copy_all()	复制所有日志到剪贴板	-	log.copy_all()
set_level(level)	动态设置过滤级别	level: 字符串 (如 'WARNING') 或整数 (0-4)	log.set_level('ERROR')
toggle()	切换组件折叠 / 展开状态 (仅 expandable=True 生效)	-	log.toggle()
update()	刷新组件显示	-	动态修改 entries 后调用

六、注意事项与最佳实践

1. 性能优化

- 大数据量日志：当日志条目超过 1000 条时，建议设置 max_lines 限制 (如 max_lines=5000)，避免 DOM 元素过多导致页面卡顿。
- 关闭自动滚动：对于非实时监控场景，可设置 auto_scroll=False，减少滚动事件带来的性能开销。

2. 样式兼容性

- 自定义 Tailwind 类时，需确保 NiceGUI 环境已加载对应的 Tailwind 样式 (NiceGUI 默认集成 Tailwind，无需额外引入)。
- 避免过度自定义样式，确保日志的可读性 (如深色背景搭配浅色文字，浅色背景搭配深色文字)。

3. 日志安全性

- 避免在日志中输出敏感信息 (如密码、Token、用户隐私数据)，防止信息泄露。
- 对于公开访问的应用，建议限制日志组件的可见性 (如仅管理员可查看)。

4. 版本兼容性

- 折叠功能 (expandable / collapsed) 仅支持 NiceGUI 2.0.0 及以上版本。
- html_id 属性仅支持 2.16.0 及以上版本。
- 若需兼容旧版本，建议避免使用新增参数，或通过 ui.run(version='x.x.x') 指定兼容版本。

总结

`ui.log` 是 NiceGUI 中轻量且功能完备的日志展示组件，通过简洁的 API 实现了日志分级、实时更新、交互操作等核心功能，无需额外前端开发即可快速集成到 Web 应用中。适用于调试输出、系统监控、操作记录等多种场景，支持样式定制和扩展（如绑定标准日志模块、日志导出），是 Python 开发者构建 Web 应用时的高效日志展示解决方案。